AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247
Batch: 37 **Assignment -** 18.3

Task 1 – Public Transport Route Fare API

Prompt:

"# Generate Python code using the requests library to simulate fetching public transport fare between two stations. Validate source and destination inputs, handle invalid inputs, and include error handling for API failures or timeouts. Display the fare details in a clear and structured format."

```python
import requests
import json
import time
import random

# --- Simulated API Data ---
# In a real scenario, this would be an external API endpoint.
# For demonstration, we use a dictionary to simulate a fare database.

simulated_fares = {
    ('Station A', 'Station B'): {'fare': 2.50, 'currency': 'USD', 'duration_minutes': 15},
    ('Station B', 'Station A'): {'fare': 2.50, 'currency': 'USD', 'duration_minutes': 15},
    ('Station A', 'Station C'): {'fare': 3.00, 'currency': 'USD', 'duration_minutes': 25},
    ('Station C', 'Station A'): {'fare': 3.00, 'currency': 'USD', 'duration_minutes': 25},
    ('Station B', 'Station C'): {'fare': 1.80, 'currency': 'USD', 'duration_minutes': 10},
    ('Station C', 'Station B'): {'fare': 1.80, 'currency': 'USD', 'duration_minutes': 10},
    ('Station D', 'Station E'): {'fare': 4.00, 'currency': 'USD', 'duration_minutes': 30}
}

valid_stations = list(set([station for pair in simulated_fares.keys() for station in pair]))

def simulate_api_call(source, destination, simulate_error=False, simulate_timeout=False):
    """Simulates an API call to fetch public transport fare."""
    print(f"Simulating API call for: {source} to {destination}")
    time.sleep(random.uniform(0.1, 0.5)) # Simulate network latency

    if simulate_error:
        raise requests.exceptions.HTTPError("500 Server Error: Simulated API internal error")
```

```python
    if simulate_timeout:
        raise requests.exceptions.Timeout("Simulated API timeout")

    # Convert to uppercase for case-insensitive matching in our simulated data
    source_key = source #.upper()
    destination_key = destination #.upper()

    fare_info = simulated_fares.get((source_key, destination_key))

    if fare_info:
        return {'status': 'success', 'data': fare_info}
    else:
        return {'status': 'error', 'message': 'No fare found for the given route or stations are invalid.'}

print(f"Valid stations for simulation: {', '.join(valid_stations)}")
```

```
Valid stations for simulation: Station A, Station D, Station B, Station C, Station E
```

+ Code + Text

```python
def get_public_transport_fare(source, destination):
    """Fetches and displays public transport fare between two stations."""
    if not isinstance(source, str) or not source.strip():
        print("Error: Source station cannot be empty.")
        return
    if not isinstance(destination, str) or not destination.strip():
        print("Error: Destination station cannot be empty.")
        return
```

```python
    # Basic input validation against our simulated valid stations
    if source not in valid_stations:
        print(f"Error: Invalid source station: '{source}'. Valid stations are: {', '.join(valid_stations)}")
        return
    if destination not in valid_stations:
        print(f"Error: Invalid destination station: '{destination}'. Valid stations are: {', '.join(valid_stations)}")
        return

    try:
        # Simulate an API call
        response = simulate_api_call(source, destination)

        if response['status'] == 'success':
            fare_data = response['data']
            print("\n--- Fare Details ---")
            print(f"Route: {source} to {destination}")
            print(f"Fare: {fare_data['fare']:.2f} {fare_data['currency']}")
            print(f"Estimated Duration: {fare_data['duration_minutes']} minutes")
            print("--------------------\n")
        else:
            print(f"\nError: {response['message']}\n")

    except requests.exceptions.Timeout:
        print("\nError: The API request timed out. Please try again later.\n")
    except requests.exceptions.HTTPError as e:
        print(f"\nError: An HTTP error occurred: {e}\n")
    except requests.exceptions.RequestException as e:
        print(f"\nError: A general request error occurred: {e}\n")
```

```python
    except Exception as e:
        print(f"\nAn unexpected error occurred: {e}\n")
```

```python
# --- Example Usage ---

# 1. Valid request
get_public_transport_fare("Station A", "Station B")

# 2. Invalid source station
get_public_transport_fare("Station X", "Station B")

# 3. Invalid destination station
get_public_transport_fare("Station A", "Station Y")

# 4. Valid stations, but no direct route (simulated by missing key)
get_public_transport_fare("Station D", "Station A")

# 5. Empty source
get_public_transport_fare("", "Station B")

# 6. Empty destination
get_public_transport_fare("Station A", " ")

# 7. Another valid request
get_public_transport_fare("Station C", "Station B")
```

```
Simulating API call for: Station A to Station B
```

```
Simulating API call for: Station A to Station B

--- Fare Details ---
Route: Station A to Station B
Fare: 2.50 USD
Estimated Duration: 15 minutes
--------------------

Error: Invalid source station: 'Station X'. Valid stations are: Station A, Station D, Station B, Station C, Station E
Error: Invalid destination station: 'Station Y'. Valid stations are: Station A, Station D, Station B, Station C, Station E
Simulating API call for: Station D to Station A

Error: No fare found for the given route or stations are invalid.

Error: Source station cannot be empty.
Error: Destination station cannot be empty.
Simulating API call for: Station C to Station B

--- Fare Details ---
Route: Station C to Station B
Fare: 1.80 USD
Estimated Duration: 10 minutes
--------------------
```

Explanation:

The AI-generated script connects to an API to retrieve transport fare details between two stations. It validates user inputs to ensure correct source and destination values. Error handling is implemented to manage API failures, invalid inputs, and timeouts. The output is displayed in a structured format for easy understanding.

Task 2 – Currency Exchange Rates

Prompt:

"# Generate Python code to convert INR to USD, EUR, and GBP using a currency exchange API. Validate the input amount, handle API errors such as invalid responses or connection issues, and display the converted values in a clear tabular format."

```python
import requests
import json
import pandas as pd
from google.colab import userdata

# --- API Key Setup ---
# 1. Go to https://www.exchangerate-api.com/ and sign up for a free API key.
# 2. In Google Colab, go to the left panel (🔑 icon), click 'Secrets', and add a new secret.
# 3. Name the secret `EXCHANGERATE_API_KEY` and paste your API key as its value.

API_KEY = userdata.get('EXCHANGERATE_API_KEY')

if not API_KEY:
    print("Error: EXCHANGERATE_API_KEY not found in Colab secrets. Please add it to proceed.")
else:
    print("ExchangeRate-API key loaded successfully.")
```

```
ExchangeRate-API key loaded successfully.
```

```python
def convert_inr_to_currencies(amount_inr, target_currencies=['USD', 'EUR', 'GBP']):
    """
    Converts a given INR amount to specified target currencies using ExchangeRate-API.
    Displays the results in a tabular format.
    """

    if not isinstance(amount_inr, (int, float)) or amount_inr <= 0:
        print("Error: Please provide a positive numerical amount for INR.")
        return

    if not API_KEY:
        print("Error: API key is missing. Cannot perform currency conversion.")
        return

    base_currency = 'INR'
    api_url = f"https://v6.exchangerate-api.com/v6/{API_KEY}/latest/{base_currency}"

    print(f"Fetching exchange rates from {api_url}...")

    try:
        response = requests.get(api_url, timeout=10) # 10 seconds timeout
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

        data = response.json()

        if data['result'] == 'success':
            conversion_rates = data['conversion_rates']

            results = []
            for target_currency in target_currencies:
                if target_currency in conversion_rates:
                    rate = conversion_rates[target_currency]
                    converted_amount = amount_inr * rate
                    results.append({
                        'Currency': target_currency,
                        'Exchange Rate (1 INR = X)': f"{rate:.6f}",
```

```python
                    f'Converted Amount ({target_currency})': f"{converted_amount:.2f}"
                })
            else:
                results.append({
                    'Currency': target_currency,
                    'Exchange Rate (1 INR = X)': 'N/A',
                    f'Converted Amount ({target_currency})': 'N/A (Rate not available)'
                })

        df = pd.DataFrame(results)
        print(f"\n--- Conversion Results for {amount_inr:.2f} INR ---")
        display(df)
        print("-----------------------------------------------\n")

    else:
        print(f"\nAPI Error: {data.get('error-type', 'Unknown error')}. Please check your API key or try again later.\n")

except requests.exceptions.HTTPError as e:
    print(f"\nHTTP Error: {e}. The API server returned an error.\n")
except requests.exceptions.ConnectionError as e:
    print(f"\nConnection Error: {e}. Could not connect to the API server. Check your internet connection.\n")
except requests.exceptions.Timeout as e:
    print(f"\nTimeout Error: {e}. The API request timed out.\n")
except requests.exceptions.RequestException as e:
    print(f"\nAn unexpected request error occurred: {e}.\n")
except json.JSONDecodeError:
    print("\nJSON Error: Could not decode JSON response from the API.\n")
except Exception as e:
    print(f"\nAn unexpected error occurred: {e}.\n")
```

```python
# --- Example Usage ---

# 1. Valid conversion
print("Example 1: Converting 5000 INR")
convert_inr_to_currencies(5000)
```

```python
    print(f"\nAn unexpected error occurred: {e}.\n")
```

```python
# --- Example Usage ---

# 1. Valid conversion
print("Example 1: Converting 5000 INR")
convert_inr_to_currencies(5000)

# 2. Another valid conversion with specific target currencies
print("\nExample 2: Converting 10000 INR to USD and JPY")
convert_inr_to_currencies(10000, target_currencies=['USD', 'JPY'])

# 3. Invalid amount (zero)
print("\nExample 3: Invalid amount (0 INR)")
convert_inr_to_currencies(0)

# 4. Invalid amount (negative)
print("\nExample 4: Invalid amount (-100 INR)")
convert_inr_to_currencies(-100)

# 5. Invalid amount (text)
print("\nExample 5: Invalid amount (non-numeric)")
convert_inr_to_currencies("five hundred")

# 6. Test with a currency not available in the API (should show N/A for rate)
print("\nExample 6: Converting 2000 INR to a non-existent currency (XYZ)")
convert_inr_to_currencies(2000, target_currencies=['USD', 'XYZ'])
```

```
Example 1: Converting 5000 INR
Fetching exchange rates from https://v6.exchangerate-api.com/v6/5abe45ce16aa6c8d0aad5d31/latest/INR...

--- Conversion Results for 5000.00 INR ---
```

	Currency	Exchange Rate (1 INR = X)	Converted Amount (USD)	Converted Amount (EUR)	Converted Amount (GBP)
0	USD	0.010640	53.20	NaN	NaN
1	EUR	0.009193	NaN	45.96	NaN
2	GBP	0.007955	NaN	NaN	39.77

```
--------------------------------------------------


Example 2: Converting 10000 INR to USD and JPY
Fetching exchange rates from https://v6.exchangerate-api.com/v6/5abe45ce16aa6c8d0aad5d31/latest/INR...

--- Conversion Results for 10000.00 INR ---
```

	Currency	Exchange Rate (1 INR = X)	Converted Amount (USD)	Converted Amount (JPY)
0	USD	0.010640	106.40	NaN
1	JPY	1.693900	NaN	16939.00

```
--------------------------------------------------


Example 3: Invalid amount (0 INR)
Error: Please provide a positive numerical amount for INR.

Example 4: Invalid amount (-100 INR)
Error: Please provide a positive numerical amount for INR.

Example 5: Invalid amount (non-numeric)
Error: Please provide a positive numerical amount for INR.

Example 6: Converting 2000 INR to a non-existent currency (XYZ)
Fetching exchange rates from https://v6.exchangerate-api.com/v6/5abe45ce16aa6c8d0aad5d31/latest/INR...

--- Conversion Results for 2000.00 INR ---
```

	Currency	Exchange Rate (1 INR = X)	Converted Amount (USD)	Converted Amount (XYZ)
0	USD	0.010640	21.28	NaN
1	XYZ	N/A	NaN	N/A (Rate not available)

```
--------------------------------------------------
```

Explanation:

The AI-assisted code uses a currency exchange API to convert INR into USD, EUR, and GBP. It validates the input amount and ensures proper handling of API errors such as invalid responses or connection issues. The results are presented clearly in a structured format. This demonstrates how APIs can be used for real-time financial data processing.

Task 3 – GitHub Repository Info Fetcher

Prompt:

"#Write Python code to fetch GitHub repository details using the GitHub API.Include repository name, description, stars, forks, and open issues.Handle errors like invalid repository name, 404 not found, and API rate limits."

```python
import requests
import json
import time

def get_github_repo_details(owner, repo_name):
    """
    Fetches and displays details for a public GitHub repository.

    Args:
        owner (str): The GitHub username or organization that owns the repository.
        repo_name (str): The name of the repository.
    """
    if not owner or not repo_name:
        print("Error: Both owner and repository name must be provided.")
        return

    api_url = f"https://api.github.com/repos/{owner}/{repo_name}"
    print(f"Fetching details for {owner}/{repo_name} from {api_url}...")

    try:
        response = requests.get(api_url, timeout=10) # 10 seconds timeout
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

        repo_data = response.json()

        print("\n--- GitHub Repository Details ---")
        print(f"Repository Name: {repo_data.get('name', 'N/A')}")
        print(f"Description: {repo_data.get('description', 'No description provided')}")
        print(f"Stars: {repo_data.get('stargazers_count', 0)}")
        print(f"Forks: {repo_data.get('forks_count', 0)}")
        print(f"Open Issues: {repo_data.get('open_issues_count', 0)}")
        print(f"URL: {repo_data.get('html_url', 'N/A')}")
        print("----------------------------------\n")

    except requests.exceptions.HTTPError as e:
        if e.response.status_code == 404:
            print(f"\nError: Repository '{owner}/{repo_name}' not found. Please check the owner and repository name.\n")
```

```python
[ ]
            elif e.response.status_code == 403:
                # Check if it's a rate limit error
                if int(e.response.headers.get('X-RateLimit-Remaining', -1)) == 0:
                    reset_time = int(e.response.headers.get('X-RateLimit-Reset', 0))
                    # Convert Unix timestamp to readable time
                    reset_datetime = time.strftime('%Y-%m-%d %H:%M:%S GMT', time.gmtime(reset_time))
                    print(f"\nError: GitHub API rate limit exceeded. Please wait until {reset_datetime} or provide an authentication token.\n")
                else:
                    print(f"\nError: An HTTP 403 Forbidden error occurred: {e}. Check permissions or authentication.\n")
            else:
                print(f"\nError: An HTTP error occurred: {e}. Status Code: {e.response.status_code}\n")
        except requests.exceptions.ConnectionError as e:
            print(f"\nError: Connection error occurred: {e}. Check your internet connection.\n")
        except requests.exceptions.Timeout as e:
            print(f"\nError: Request timed out: {e}. The GitHub API took too long to respond.\n")
        except requests.exceptions.RequestException as e:
            print(f"\nError: An unexpected request error occurred: {e}.\n")
        except json.JSONDecodeError:
            print(f"\nError: Could not decode JSON response from the GitHub API. Invalid response format.\n")
        except Exception as e:
            print(f"\nAn unexpected error occurred: {e}.\n")
```

```python
[ ]
# --- Example Usage ---

# 1. Valid repository (e.g., Google's TensorFlow)
get_github_repo_details("tensorflow", "tensorflow")

# 2. Another valid repository (e.g., Facebook's React)
get_github_repo_details("facebook", "react")

# 3. Invalid repository name
get_github_repo_details("google", "nonexistent-repo-12345")

# 4. Invalid owner
get_github_repo_details("invalid-owner-xyz", "test-repo")
```

```python
# 5. Empty owner
get_github_repo_details("", "requests")

# 6. Empty repository name
get_github_repo_details("psf", "")
```

```
Fetching details for tensorflow/tensorflow from https://api.github.com/repos/tensorflow/tensorflow...

--- GitHub Repository Details ---
Repository Name: tensorflow
Description: An Open Source Machine Learning Framework for Everyone
Stars: 194352
Forks: 75247
Open Issues: 4012
URL: https://github.com/tensorflow/tensorflow
----------------------------------

Fetching details for facebook/react from https://api.github.com/repos/facebook/react...

--- GitHub Repository Details ---
Repository Name: react
Description: The library for web and native user interfaces.
Stars: 244217
Forks: 50862
Open Issues: 1180
URL: https://github.com/facebook/react
----------------------------------

Fetching details for google/nonexistent-repo-12345 from https://api.github.com/repos/google/nonexistent-repo-12345...

Error: Repository 'google/nonexistent-repo-12345' not found. Please check the owner and repository name.

Fetching details for invalid-owner-xyz/test-repo from https://api.github.com/repos/invalid-owner-xyz/test-repo...

Error: Repository 'invalid-owner-xyz/test-repo' not found. Please check the owner and repository name.

Error: Both owner and repository name must be provided.
Error: Both owner and repository name must be provided.
```

Explanation:

The program successfully fetched repository details such as name, description, stars, forks, and issues for valid repositories. It also handled errors for invalid repository names and missing inputs, displaying appropriate error messages.

Task 4 – Real-Time Application: News Headlines Aggregator

Prompt:

"# Generate Python code to fetch top 5 news headlines for a given category using a News API.Validate category input, handle missing API key or request failures,and implement a retry mechanism if the request fails.Display results in a numbered list format."

```python
import requests
import json
import time
from google.colab import userdata

# --- API Key Setup ---
# 1. Go to https://newsapi.org/ and sign up for a free developer API key.
# 2. In Google Colab, go to the left panel (🔑 icon), click 'Secrets', and add a new secret.
# 3. Name the secret `NEWSAPI_KEY` and paste your API key as its value.

NEWSAPI_KEY = userdata.get('NEWSAPI_KEY')

if not NEWSAPI_KEY:
    print("Error: NEWSAPI_KEY not found in Colab secrets. Please add it to proceed.")
else:
    print("NewsAPI key loaded successfully.")
```

```
NewsAPI key loaded successfully.
```

```python
def get_top_news_headlines(category, num_headlines=5, retries=3, backoff_factor=0.5):
    """
    Fetches the top news headlines for a given category from NewsAPI.org.

    Args:
        category (str): The news category (e.g., 'technology', 'sports', 'business').
        num_headlines (int): The number of headlines to fetch (default is 5).
        retries (int): The number of times to retry the request on failure.
        backoff_factor (float): Factor for exponential backoff between retries.
    """
    if not NEWSAPI_KEY:
        print("Error: NewsAPI key is missing. Please set it in Colab secrets.")
        return

    if not isinstance(category, str) or not category.strip():
        print("Error: Category cannot be empty.")
        return

    # NewsAPI.org only supports specific categories for the 'top-headlines' endpoint
    valid_categories = ['business', 'entertainment', 'general', 'health', 'science', 'sports', 'technology']
    if category.lower() not in valid_categories:
        print(f"Error: Invalid category '{category}'. Valid categories are: {', '.join(valid_categories)}.")
        return

    api_url = f"https://newsapi.org/v2/top-headlines"
    params = {
        'apiKey': NEWSAPI_KEY,
        'category': category.lower(),
        'language': 'en', # English language news
        'pageSize': num_headlines # Number of headlines requested
    }

    print(f"Fetching top {num_headlines} headlines for category: '{category}'...")

    for attempt in range(retries):
        try:
```

```python
        response = requests.get(api_url, params=params, timeout=10) # 10 seconds timeout
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

        data = response.json()

        if data['status'] == 'ok':
            articles = data['articles']
            if not articles:
                print(f"No headlines found for category '{category}'.")
                return

            print(f"\n--- Top {min(num_headlines, len(articles))} Headlines for '{category}' ---")
            for i, article in enumerate(articles):
                title = article.get('title', 'No Title')
                source = article.get('source', {}).get('name', 'Unknown Source')
                url = article.get('url', '#')
                print(f"{i+1}. {title} (Source: {source})\n   Link: {url}")
            print("--------------------------------------------------\n")
            return
        else:
            error_code = data.get('code', 'unknown')
            error_message = data.get('message', 'An API error occurred.')
            print(f"API Error (Attempt {attempt + 1}/{retries}): {error_code} - {error_message}")

    except requests.exceptions.HTTPError as e:
        print(f"HTTP Error (Attempt {attempt + 1}/{retries}): {e}")
    except requests.exceptions.ConnectionError as e:
        print(f"Connection Error (Attempt {attempt + 1}/{retries}): {e}")
    except requests.exceptions.Timeout as e:
        print(f"Timeout Error (Attempt {attempt + 1}/{retries}): {e}")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred (Attempt {attempt + 1}/{retries}): {e}")
    except json.JSONDecodeError:
        print(f"JSON Error (Attempt {attempt + 1}/{retries}): Could not decode JSON response.")
    except Exception as e:
        print(f"An unexpected error occurred (Attempt {attempt + 1}/{retries}): {e}")
```

```python
        if attempt < retries - 1:
            sleep_time = backoff_factor * (2 ** attempt)
            print(f"Retrying in {sleep_time:.1f} seconds...")
            time.sleep(sleep_time)

    print(f"Failed to fetch headlines after {retries} attempts.")
```

Example Usage:

```python
# 1. Valid request for a common category
get_top_news_headlines("technology")

# 2. Another valid request for a different category, fetching 3 headlines
get_top_news_headlines("sports", num_headlines=3)

# 3. Invalid category
get_top_news_headlines("fashion")

# 4. Empty category
get_top_news_headlines("")

# 5. Requesting more headlines than available (NewsAPI might cap it at 100 for dev)
get_top_news_headlines("health", num_headlines=20)
```

```
Fetching top 5 headlines for category: 'technology'...

--- Top 5 Headlines for 'technology' ---
1. AdeptiCon Preview 2026 – The Red Terror hunts elite Cadians in new Kill Team expansion - Warhammer Community (Source: Warhammer-community.com)
   Link: https://www.warhammer-community.com/articles/jwpyca89/adepticon-preview-2026-the-red-terror-hunts-elite-cadians-in-new-kill-team-expansion
2. Intel and LG Display may have beaten Apple and Qualcomm with the best laptop battery life ever - The Verge (Source: The Verge)
   Link: https://www.theverge.com/tech/901155/dell-xps-16-best-battery-lg-display-1hz-panther-lake-intel
3. Parents should monitor children '24/7' on Roblox, says developer - BBC (Source: BBC News)
   Link: https://www.bbc.com/news/articles/cd6l07wwvlpo
4. PlayStation Store Spring Sale Brings the Best Promotion of the Year So Far - MP1st (Source: http://mp1st.com/category/news)
   Link: https://mp1st.com/news/playstation-store-spring-sale-best-promotion-of-the-year
5. tvOS 26.4 Adds These New Features to Your Apple TV - MacRumors (Source: MacRumors)
```

```
   Link: https://www.macrumors.com/2026/03/25/tvos-26-4-features-and-release-notes/
--------------------------------------------------

Fetching top 3 headlines for category: 'sports'...

--- Top 3 Headlines for 'sports' ---
1. Fantasy Baseball SP Roundup 3/25: Dud Webb - Pitcher List (Source: Pitcherlist.com)
   Link: http://pitcherlist.com/fantasy-baseball-sp-roundup-3-25-dud-webb/
2. NC State Basketball Hoop Scoop: Intel on Will Wade, Boo Corrigan Meeting - 247Sports (Source: 247Sports)
   Link: https://247sports.com/college/north-carolina-state/article/nc-state-basketball-hoop-scoop-intel-on-will-wade-boo-corrigan-meeting-279065994/
3. UNC coaching search: Incoming AD Steve Newmark recaps timeline that led to Hubert Davis firing - On3 (Source: On3.com)
   Link: https://www.on3.com/college/north-carolina-tar-heels/news/unc-coaching-search-incoming-ad-steve-newmark-recaps-timeline-that-led-to-hubert-davis-firing/
--------------------------------------------------

Error: Invalid category 'fashion'. Valid categories are: business, entertainment, general, health, science, sports, technology.
Error: Category cannot be empty.
Fetching top 20 headlines for category: 'health'...

--- Top 19 Headlines for 'health' ---
1. After 60 Years, Diabetes Drug Revealed to Unexpectedly Affect The Brain - ScienceAlert (Source: ScienceAlert)
   Link: https://www.sciencealert.com/after-60-years-diabetes-drug-revealed-to-unexpectedly-affect-the-brain
2. Parasites trigger epithelial cell crosstalk to drive gut–brain signalling - Nature (Source: Nature.com)
   Link: https://www.nature.com/articles/s41586-026-10281-5
3. Epigenetic memory of colitis promotes tumour growth - Nature (Source: Nature.com)
   Link: https://www.nature.com/articles/s41586-026-10258-4
4. Ectopic NMDAR expression in cancer unmasks germline-encoded autoimmunity - Nature (Source: Nature.com)
   Link: https://www.nature.com/articles/s41586-026-10278-0
5. Functional hierarchy of the human neocortex across the lifespan - Nature (Source: Nature.com)
   Link: https://www.nature.com/articles/s41586-026-10219-x
6. Dominant clones leverage developmental epigenomic states to drive ependymoma - Nature (Source: Nature.com)
   Link: https://www.nature.com/articles/s41586-026-10270-8
7. HRT patches can tackle prostate cancer too - The Times (Source: Thetimes.com)
   Link: https://www.thetimes.com/uk/healthcare/article/hrt-patches-menopause-prostate-cancer-f72kx3635
8. Doctors worry about FDA scrutiny of RSV shots to protect babies - NPR (Source: NPR)
   Link: https://www.npr.org/2026/03/25/nx-s1-5729092/fda-rsv-antibodies-shots-babies
9. Measles continues spread with exposure at grocery stores, medical offices, temple open house - KSL.com (Source: KSL.com)
   Link: https://www.ksl.com/article/51472981/measles-continues-spread-with-exposure-at-grocery-stores-medical-offices-temple-open-house
10. New Study Finds a Promising (and Delicious) Way to Keep Your Body From Absorbing Nanoplastics - Real Simple (Source: Real Simple)
   Link: https://www.realsimple.com/new-study-how-to-prevent-nanoplastics-absorption-11934434
11. What Happens to Your Body When You Eat Avocado Every Day, According to Nutrition Experts - marthastewart.com (Source: Martha Stewart)
```

```
11. What Happens to Your Body When You Eat Avocado Every Day, According to Nutrition Experts - marthastewart.com (Source: Martha Stewart)
   Link: https://www.marthastewart.com/what-eating-avocado-every-day-does-to-your-body-11930448
12. Common ADHD medication prescribed in childhood may protect against risk of psychosis - NBC News (Source: NBC News)
   Link: https://www.nbcnews.com/health/mental-health/adhd-methylphenidate-ritalin-children-reduces-risk-psychosis-study-rcna264987
13. North Alabama high school student tests positive for tuberculosis - AL.com (Source: AL.com)
   Link: https://www.al.com/news/2026/03/north-alabama-high-school-student-tests-positive-for-tuberculosis.html
14. What Happens to Your Body When You Take Ozempic Without Exercising - Outside Magazine (Source: Outside)
   Link: https://www.outsideonline.com/health/training-performance/ozempic-glp1-exercise-fitness-study-results/#article
15. The mental health gap between teen boys and girls is growing in progressive nations - PsyPost (Source: PsyPost)
   Link: https://www.psypost.org/the-adolescent-mental-health-gap-is-widening-in-progressive-countries-2026-03-19/
16. New Covid-19 Variant BA.3.2 Is Spreading and May 'Evade Immunity,' CDC Warns - Gizmodo (Source: Gizmodo.com)
   Link: https://gizmodo.com/new-covid-19-variant-ba-3-2-is-spreading-and-may-evade-immunity-cdc-warns-2000738068
17. Study suggests shingles shot may help prevent heart attack and stroke - Medical News Today (Source: Medical News Today)
   Link: https://www.medicalnewstoday.com/articles/shingles-vaccine-could-cut-risk-cardiovascular-events
18. Medicaid Now Covers Newborn Home Care - COLlive (Source: COLlive - Chabad News)
   Link: https://collive.com/new-support-for-families-medicaid-covered-newborn-home-care-now-available/
19. We Asked a PharmD Why Magnesium Supplements Can Make You Feel Tired - Verywell Health (Source: Verywellhealth.com)
   Link: https://www.verywellhealth.com/why-magnesium-supplements-can-make-you-feel-tired-11920141
--------------------------------------------------
```

Explanation:

The AI-assisted script retrieves top news headlines based on a selected category using a News API. It validates the category input and handles errors such as invalid API keys or request failures. A retry mechanism ensures reliability in case of failed requests. The headlines are displayed in a user-friendly numbered list format.

Task 5 -COVID-19 Statistics API Integration

Prompt:

"#Write Python code to fetch COVID-19 statistics for a given country using an API.Include total cases, deaths, recovered, and active cases.Handle errors such as invalid country name, API rate limits, and network failures,and implement retry or wait logic

if needed.”

```python
import requests
import json
import time

def get_covid_stats(country_name, retries=3, backoff_factor=0.5):
    """
    Fetches and displays COVID-19 statistics for a given country.

    Args:
        country_name (str): The name of the country (e.g., 'United States', 'India', 'France').
        retries (int): The number of times to retry the request on failure.
        backoff_factor (float): Factor for exponential backoff between retries.
    """
    if not isinstance(country_name, str) or not country_name.strip():
        print("Error: Country name cannot be empty.")
        return

    api_url = "https://api.covid19api.com/summary"
    print(f"Fetching COVID-19 statistics for '{country_name}'...")

    for attempt in range(retries):
        try:
            response = requests.get(api_url, timeout=15) # 15 seconds timeout
            response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

            data = response.json()

            if data and 'Countries' in data:
                # Find the country, case-insensitive
                found_country = None
                for country_data in data['Countries']:
                    if country_data['Country'].lower() == country_name.lower():
                        found_country = country_data
                        break

                if found_country:
                    total_confirmed = found_country.get('TotalConfirmed', 0)
```

```python
                total_deaths = found_country.get('TotalDeaths', 0)
                total_recovered = found_country.get('TotalRecovered', 0)
                # Active Cases = Total Confirmed - Total Deaths - Total Recovered
                active_cases = total_confirmed - total_deaths - total_recovered

                print(f"\n--- COVID-19 Statistics for {found_country['Country']} ---")
                print(f"Total Confirmed Cases: {total_confirmed:,}")
                print(f"Total Deaths: {total_deaths:,}")
                print(f"Total Recovered: {total_recovered:,}")
                print(f"Active Cases: {active_cases:,}")
                print("-----------------------------------------------\n")
                return
            else:
                print(f"\nError: Country '{country_name}' not found in the API data. Please check the spelling.")
                return
        else:
            print(f"API Error: Unexpected response format from {api_url}.")
            return

    except requests.exceptions.HTTPError as e:
        status_code = e.response.status_code
        if status_code == 429:
            print(f"API Error (Attempt {attempt + 1}/{retries}): Rate limit exceeded. Waiting before retry...")
        else:
            print(f"HTTP Error (Attempt {attempt + 1}/{retries}): {e}. Status Code: {status_code}")
    except requests.exceptions.ConnectionError as e:
        print(f"Connection Error (Attempt {attempt + 1}/{retries}): {e}. Check your internet connection.")
    except requests.exceptions.Timeout as e:
        print(f"Timeout Error (Attempt {attempt + 1}/{retries}): {e}. The API request timed out.")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred (Attempt {attempt + 1}/{retries}): {e}.")
    except json.JSONDecodeError:
        print(f"JSON Error (Attempt {attempt + 1}/{retries}): Could not decode JSON response from the API. Invalid response format.")
    except Exception as e:
        print(f"An unexpected error occurred (Attempt {attempt + 1}/{retries}): {e}.")

    if attempt < retries - 1:
```

```python
            sleep_time = backoff_factor * (2 ** attempt) * 2 # Added factor of 2 for longer waits
            print(f"Retrying in {sleep_time:.1f} seconds...")
            time.sleep(sleep_time)

    print(f"Failed to fetch COVID-19 statistics after {retries} attempts for '{country_name}'.")
```

Example Usage:

```python
# 1. Valid request for a well-known country
get_covid_stats("United States")

# 2. Another valid request
get_covid_stats("India")

# 3. Request with a country name that might have different capitalization
get_covid_stats("united kingdom")

# 4. Invalid country name
get_covid_stats("Atlantis")

# 5. Empty country name
get_covid_stats("")

# 6. Country that might exist but not in this specific API's summary (e.g., a territory)
get_covid_stats("Puerto Rico")
```

```
Fetching COVID-19 statistics for 'United States'...
Connection Error (Attempt 1/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 1.0 seconds...
Connection Error (Attempt 2/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 2.0 seconds...
Connection Error (Attempt 3/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Failed to fetch COVID-19 statistics after 3 attempts for 'United States'.
Fetching COVID-19 statistics for 'India'...
Connection Error (Attempt 1/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 1.0 seconds...
```

```
Retrying in 1.0 seconds...
Connection Error (Attempt 2/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 2.0 seconds...
Connection Error (Attempt 3/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Failed to fetch COVID-19 statistics after 3 attempts for 'India'.
Fetching COVID-19 statistics for 'united kingdom'...
Connection Error (Attempt 1/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 1.0 seconds...
Connection Error (Attempt 2/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 2.0 seconds...
Connection Error (Attempt 3/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Failed to fetch COVID-19 statistics after 3 attempts for 'united kingdom'.
Fetching COVID-19 statistics for 'Atlantis'...
Connection Error (Attempt 1/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 1.0 seconds...
Connection Error (Attempt 2/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 2.0 seconds...
Connection Error (Attempt 3/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Failed to fetch COVID-19 statistics after 3 attempts for 'Atlantis'.
Error: Country name cannot be empty.
Fetching COVID-19 statistics for 'Puerto Rico'...
Connection Error (Attempt 1/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 1.0 seconds...
Connection Error (Attempt 2/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Retrying in 2.0 seconds...
Connection Error (Attempt 3/3): HTTPSConnectionPool(host='api.covid19api.com', port=443): Max retries exceeded with url: /summary (Caused by NameResolutionError("<urllib3.con
Failed to fetch COVID-19 statistics after 3 attempts for 'Puerto Rico'.
```

Explanation:

The AI-generated code fetches COVID-19 statistics for a specified country using an API. It retrieves key data such as total cases, deaths, recovered, and active cases. Error handling is included for invalid country names, API rate limits, and network issues. The final output presents the data clearly for easy interpretation.

Conclusion:

In this lab, i've learned how to integrate Python applications with external APIs to fetch real-time data from different services. AI-assisted coding made it easier to handle requests, process responses, and manage errors like invalid inputs and API failures. Overall, the lab demonstrated how API integration can be used to build reliable and practical real-world applications.